

HANGFUL

Software Requirements Specification (SRS)

CMSI 4072: Senior Project II
Loyola Marymount University

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for the Hangful mobile application. Hangful is a social utility iOS app designed to reduce friction in planning casual hangouts among Gen Z users (ages 18–28) by combining hangout scheduling with a deal and place discovery layer. This document serves as the authoritative reference for the development, testing, and validation of the Hangful MVP.

1.2 Scope

The Hangful MVP consists of a native iOS application built with Swift 5.9 and SwiftUI. The application enables users to create hangouts at specific places with optional brand-sponsored deals, share hangout links with friends, coordinate via a friends list and availability calendar, upload proof of attendance, and receive redemption codes as rewards. The MVP uses mock data for all backend interactions, with architecture designed for future integration with a Node.js/Express API and PostgreSQL database on Supabase.

1.3 Definitions and Acronyms

Term	Definition
Hangout	A scheduled social gathering at a specific place, organized by a host and shared with friends via a unique link
Deal	A promotional offer from a local brand attached to a place, redeemable after proof of hangout attendance
Place	A physical location (restaurant, coffee shop, outdoor venue, etc.) where a hangout can occur
Drop	A time-limited release of deals (deferred beyond MVP; deals are always visible in current scope)
Proof	A photograph uploaded by a user as evidence of hangout attendance at the designated place
Redemption Code	An 8-character cryptographically random alphanumeric code issued upon proof approval
Quick Hangout	A category shortcut (Movie, Coffee, Matcha, Beach, Arcade) that pre-fills the hangout creation flow
Best Friends	A computed list of the top 5 friends ranked by hangout frequency, displayed on the Friends tab
OTP	One-Time Password delivered via SMS for phone-based authentication
MVVM	Model-View-ViewModel architectural pattern used by the iOS application
MVP	Minimum Viable Product; the semester-deliverable scope of the application

1.4 References

Hangful Product Requirements Document (PRD) and Technical Requirements Document (TRD), Version 1.0, January 2026. Apple Human Interface Guidelines for iOS 17. CMSI 4072 Senior Project II course requirements, LMU Department of Computer Science.

1.5 Overview

Section 2 provides a general description of the product including user characteristics and constraints. Section 3 specifies all functional requirements organized by feature area. Section 4 specifies non-functional requirements including performance, security, and usability. Section 5 defines the system interfaces.

2. General Description

2.1 Product Perspective

Hangful occupies a unique space between event-focused platforms (Partiful, Eventbrite) and passive social media (Instagram, TikTok). Unlike event platforms that target large gatherings, Hangful focuses on casual, close-friend, daily plan-making. Unlike social media that competes for attention online, Hangful incentivizes real-world interaction through brand-sponsored deals. The product is a standalone iOS mobile application with no dependency on external social platforms.

2.2 User Characteristics

The primary user is a Gen Z or young millennial (18–28) who wants to hang out more frequently with close friends but finds coordination logistically difficult, especially post-college when schedules become complex. Users are mobile-native, comfortable with social apps, and motivated by both social connection and tangible rewards (discounts, freebies). The secondary user is a local brand manager seeking authentic Gen Z engagement, but this persona is deferred beyond the MVP (brand management is handled via database seeding).

2.3 Constraints

The application requires iOS 17.0 or later and is developed as a solo project within a single academic semester. The MVP uses mock data rather than a live backend, with the architecture designed to support real API integration by replacing the MockData service class. The application does not include a web-based Brand Portal, messaging system, or real-time location sharing in the MVP scope.

2.4 Assumptions and Dependencies

The application assumes users have an iPhone with a rear-facing camera for proof photos, cellular or Wi-Fi connectivity for future API communication, and a valid phone number capable of receiving SMS for OTP authentication. The mock data environment assumes a user named Jaime based in the Los Angeles area with 9 seeded friends, 9 places, and 4 active deals.

3. Functional Requirements

3.1 Authentication

ID	Requirement
FR-001	The system shall allow users to authenticate via phone number and 6-digit one-time password (OTP)
FR-002	The system shall validate phone numbers in E.164 format with a minimum of 10 digits
FR-003	The system shall allow navigation back from OTP entry to phone number entry
FR-004	The system shall display an error message for invalid OTP codes
FR-005	The system shall persist authentication state so users remain logged in across app launches (future: Keychain)

3.2 Hangout Management

ID	Requirement
FR-010	The system shall display a home feed of upcoming hangouts sorted by date ascending
FR-011	The system shall display hangout cards with title, date/time, place name, attendee names, and status chip (confirmed, pending, cancelled)
FR-012	The system shall provide a 5-step hangout creation flow: Name → Place Selection → Date/Time → Confirmation → Share Link
FR-013	The system shall generate a unique shareable link (hangful.app/h/{code}) for each created hangout
FR-014	The system shall provide Quick Hangout category shortcuts (Movie, Coffee, Matcha, Beach, Arcade) that pre-fill the hangout title
FR-015	The system shall display a copy button and native share sheet for hangout links
FR-016	The system shall display deal badges on places that have active deals during the place selection step
FR-017	The system shall automatically attach a place's active deal to the hangout when the user selects that place
FR-018	The system shall categorize past hangouts into 'Needs Action' (proof needed) and 'Completed' (redeemed) sections
FR-019	The system shall display a 'cancelled' state for cancelled hangouts with appropriate visual treatment

3.3 Proof Submission and Redemption

ID	Requirement
----	-------------

FR-020	The system shall allow users to select a photo from their device photo library as proof of hangout attendance
FR-021	The system shall display a preview of the selected proof photo before submission
FR-022	The system shall show an upload progress indicator during proof submission
FR-023	The system shall transition the hangout status to 'Pending Review' after proof submission
FR-024	Upon proof approval, the system shall display an 8-character alphanumeric redemption code
FR-025	The system shall provide a copy-to-clipboard button for redemption codes
FR-026	The system shall display the associated business name alongside the redemption code for easy reference

3.4 Friends System

ID	Requirement
FR-030	The system shall display a friends list with profile avatar, display name, location, and last seen context (e.g., '2 days ago (Matcha @ Santa Monica Pier)')
FR-031	The system shall display a 'Best Friends' horizontal scroll row showing the top 5 friends ranked by hangout frequency
FR-032	The system shall support a Friends/Groups tab toggle
FR-033	The system shall display friend groups with group emoji, name, member count, and stacked member avatars
FR-034	The system shall display a friend profile view with bio, hangout count together, mutual friend count, location, and last seen context
FR-035	The system shall provide Star (favorite), Friends (status), and Message action buttons on friend profiles
FR-036	The system shall display an inline friend request banner with Accept and Decline actions
FR-037	The system shall allow adding friends by search, phone number, or referral code
FR-038	The system shall display a weekly availability row on friend profiles
FR-039	The system shall support friend removal with a confirmation dialog
FR-040	The system shall display a search bar at the bottom of the friends list for filtering by name

3.5 Calendar and Availability

ID	Requirement
FR-050	The system shall display a Calendar tab with a Weekly View / Monthly View segmented toggle
FR-051	The system shall display a Mon–Sun day selector showing the current week's dates

FR-052	The system shall highlight the current day in the day selector
FR-053	The system shall display green availability bars under days where at least one friend is available
FR-054	The system shall display an 'Available Friends' list for the selected day with availability time windows
FR-055	Each friend entry shall show a green 'available' or red 'not available' badge
FR-056	Unavailable friends shall display their next available day and time (e.g., 'Friday @ 10am')
FR-057	Each friend entry shall display a colored progress bar (green for available, red for unavailable)

3.6 Friend Map

ID	Requirement
FR-060	The system shall display a MapKit map with friend profile avatars as annotation pins
FR-061	The system shall center the map on the Los Angeles area by default
FR-062	The system shall display '+ Hangout Spot' and 'Show Hangout Recs' action buttons below the map
FR-063	The system shall display a friends list below the map with avatar, name, location, and last seen context
FR-064	The system shall be accessible from the Friends tab via a map icon in the navigation toolbar

3.7 Place and Deal Discovery

ID	Requirement
FR-070	The system shall display an Explore tab with a searchable list of places
FR-071	The system shall support filtering places by category (Food & Drink, Coffee, Entertainment, Outdoors, Shopping)
FR-072	The system shall support a 'Deals only' toggle that filters to places with active deals
FR-073	The system shall display a place detail view with category icon, name, address, and active deal information
FR-074	Deal information shall include title, description, reward type emoji, reward value, remaining quantity, and expiration

3.8 Referral System

ID	Requirement
----	-------------

FR-080	The system shall gate the referral feature behind completion of the user's first hangout proof approval
FR-081	When unlocked, the system shall display a shareable referral link (hangful.app/r/{code}) with a native share sheet
FR-082	When locked, the system shall display a message explaining how to unlock referrals

3.9 Profile and Brand Requests

ID	Requirement
FR-090	The system shall display the user's phone number and referral code on the Profile tab
FR-091	The system shall allow users to submit brand name suggestions via a brand request form
FR-092	The system shall provide a logout function with a confirmation dialog

4. Non-Functional Requirements

4.1 Performance

The application shall render the home feed within 1 second of tab selection. Mock data simulates network latency with 500–1500ms delays to validate that loading states and progress indicators function correctly under realistic conditions. The photo picker and proof submission flow shall display a progress indicator and complete within 3 seconds on a standard Wi-Fi connection when connected to a real backend.

4.2 Usability

The application shall follow Apple's Human Interface Guidelines for iOS 17, including standard tab bar navigation, navigation stack management, and sheet presentation. All interactive elements shall have a minimum tap target of 44x44 points. The color scheme uses orange as the primary accent color with purple for friend names, matching the established brand identity from the design mockups. The application shall support Dynamic Type for accessibility.

4.3 Reliability

The application shall handle all error states gracefully with user-facing error messages and retry options. The authentication flow shall validate input before making network requests. The hangout creation flow shall validate that a title and place are selected before allowing submission. The application shall not crash on empty states (no hangouts, no friends, no deals).

4.4 Security

In the production version, JWT tokens shall be stored in the iOS Keychain using the `kSecAttrAccessibleWhenUnlockedThisDeviceOnly` attribute. OTP requests shall be rate-limited to 3 per phone number per 10 minutes. Redemption codes shall be generated using cryptographically secure random bytes. No user credentials shall be stored in `UserDefaults` or logged to console in production builds.

4.5 Portability

The application targets iOS 17.0 and later, supporting all iPhones from iPhone SE (2nd generation) onwards. The MVVM architecture with a centralized `MockData` service class enables backend portability: replacing `MockData` with a real `APIClient` class requires no changes to Views or the majority of ViewModel logic.

4.6 Maintainability

The codebase follows the MVVM architectural pattern with clear separation between Models (data structures), ViewModels (state management), Views (UI), Services (data access), and Components (reusable UI elements). The 10-file structure is organized into logical groups that map directly to feature areas, enabling independent development and testing of each feature.

5. System Interfaces

5.1 User Interface

The application presents a four-tab navigation interface: Home (hangout feed with Quick Hangouts), Calendar (friend availability), Friends (friend list, groups, map), and Profile (referral, brand requests, account). Navigation within each tab uses SwiftUI NavigationStack for hierarchical drill-down. Modal flows (hangout creation, add friend, brand request) use SwiftUI sheet presentation. The authentication flow is presented as a full-screen replacement of the tab interface, gated by the AuthViewModel's authState property.

5.2 Hardware Interfaces

The application requires a touchscreen display for all user interactions, a rear-facing camera (accessed via PhotosPicker and UIImagePickerController) for proof photo capture, and network connectivity (cellular or Wi-Fi) for future API communication. The MapKit framework accesses the device's GPS for map centering (currently defaulting to the Los Angeles area).

5.3 Software Interfaces

The application uses the following Apple frameworks: SwiftUI (UI rendering), Combine (reactive state management via @Published properties), MapKit (friend map display), PhotosUI (photo library access via PhotosPicker), UIKit (UIImage for proof photo handling, UIPasteboard for clipboard operations). In the production version, the application will interface with a Node.js/Express backend API over HTTPS/REST with JSON payloads and JWT Bearer token authentication.